

Creating a Non-Commutative Polynomial Sum-of-Squares Solver

Brian An

August 29, 2025

Abstract

Non-commutative polynomials, specifically matrix-valued polynomials, arise in quantum information when representing the win probability of nonlocal games. By finding the sum-of-squares (SOS) decomposition of a game polynomial, we can get a bound on the maximum win probability of a nonlocal game. We can reformulate the problem of finding the SOS decomposition of a polynomial as a hierarchy of semidefinite programs (SDP), which can be solved efficiently using convex solvers in Python or other languages. In this paper we chronicle the development of our non-commutative polynomial sum-of-squares solver in Python and the challenges encountered along the way.

Contents

1	Introduction	2
1.1	Overview	2
1.2	Motivation: Nonlocal games	2
1.3	Sum-of-squares	3
1.4	Semidefinite Optimization	3
2	Creating the solver	4
2.1	Commutative Unconstrained Solver	5
2.2	Commutative Constrained Solver	6
2.3	Commutative Complex Unconstrained Solver	8
2.4	Non-Commutative Unconstrained Solver	9
2.5	Non-Commutative Constrained Solver	10
3	Areas of Improvement	13
4	Acknowledgements	13

1 Introduction

1.1 Overview

A nonlocal game describes a classical verifier playing a game with two separated but possibly entangled players [9]. If the players share a quantum state during the game, then the nonlocal game is a quantum nonlocal game; otherwise, it is a classical nonlocal game. Nonlocal games are studied partially due to interest in how they are affected by quantum states. The probability of winning a quantum nonlocal game can be represented as a matrix-valued polynomial optimization. By getting a bound of the value of this polynomial, we can find the maximum chance of winning a nonlocal game. If a polynomial has a sum-of-squares (SOS) decomposition, that is a certificate of the non-negativity of the polynomial. By finding the minimum λ such that $\lambda - p$ has an SOS decomposition, where p is a polynomial, we can effectively get a bound on the maximum of p , and hence the maximum probability of winning a nonlocal game. The problem of finding the SOS decomposition of $\lambda - p$ can be formulated as a hierarchy of semidefinite programs (SDP), a tool commonly used in quantum information [11]. SDPs can generally be solved efficiently using convex solvers such as MOSEK, SCS, CVXOPT in Python. This report delves into the motivation behind our solver, the tools used, and the process of developing our solver, starting with a commutative, unconstrained solver and finishing with a non-commutative, constrained solver. The files pertaining to the solver can be found at <https://github.com/brianan45/NCSOSsolver>.

1.2 Motivation: Nonlocal games

Consider the CHSH game [3]: the game has a referee and two players, Alice and Bob. The referee sends one bit $x \in \{0, 1\}$ to Alice and one bit $y \in \{0, 1\}$ to Bob. In return, Alice sends a bit $a \in \{0, 1\}$ to the referee and Bob sends a bit $b \in \{0, 1\}$ to the referee. If $a + b = xy \pmod 2$, then Alice and Bob win. If we consider the classical version of the game where Alice and Bob share only classical resources, the maximum win probability is 75%. However, if Alice and Bob share a quantum state, the maximum win probability is $\sim 85\%$ [2].

The win probability of the nonlocal CHSH game is represented by the polynomial $p = X_0Y_0 + X_1Y_0 + X_0Y_1 - X_1Y_1$, where $X_i^* = X_i, Y_i^* = Y_i, X_i^2 = I, Y_i^2 = I, Y_iX_j = X_jY_i \forall i, j \in \{0, 1\}$. For a sufficiently large λ , we can rewrite $\lambda I - p$ as a sum of squares times the matrix constraints, which acts as a certificate of non-negativity. By finding the minimum λ such that p can be written in this form, we effectively find the minimum of p , from which we can find the maximum winning probability of the nonlocal CHSH game; this technique can be applied to other nonlocal games.

1.3 Sum-of-squares

Definition 1.1. A polynomial $p \in \mathbb{R}[x_1, \dots, x_n]$ is **nonnegative** if $p(x_1, \dots, x_n) \geq 0 \forall x_1, \dots, x_n \in \mathbb{R}$

Definition 1.2. A polynomial $p \in \mathbb{R}[x_1, \dots, x_n]$ is a **sum-of-squares (SOS)** if there exist polynomials $q_1, \dots, q_k \in \mathbb{R}[x_1, \dots, x_n]$ such that $p = \sum_{i=1}^k q_i^2$; we call $\sum_{i=1}^k q_i^2$ the **sum-of-squares decomposition** of p

As sums-of-squares are non-negative, the SOS decomposition of p acts as a certificate of non-negativity of p . Therefore, by finding the minimum λ such that $p + \lambda$ is an SOS, we effectively find the minimum of p ; for example, if $p = x^2 - 4$, the minimum λ such that p is an SOS is 4, indicating that $-\lambda = -4$ is the minimum of p .

SOS polynomials have longtime been a subject of interest in mathematics. In 1900, Hilbert posed the question of whether a multivariate non-negative real polynomial always has an SOS decomposition in rational functions; this question is known as the 17th Hilbert problem. The answer was proven to be no, with the counterexample being the Motzkin polynomial $M(x, y) = x^4y^2 + x^2y^4 + 1 - 3x^2y^2$, which is non-negative but has no SOS decomposition in rational functions [7]; however, the Motzkin polynomial is a rational SOS, as is every non-negative real polynomial. It is known that any non-negative univariate polynomial of even degree has an SOS decomposition [7]. A non-negative multivariate polynomial is guaranteed to have an SOS decomposition if the polynomial is of degree 2 or if the polynomial is in two variables and up to degree 4 [7].

1.4 Semidefinite Optimization

Definition 1.3. A **positive semidefinite (PSD)** matrix Q is a symmetric matrix satisfying $\vec{v}^T Q \vec{v} \geq 0 \forall \vec{v}$. We denote Q is PSD using the notation $Q \succeq 0$.

Definition 1.4. A **semidefinite program (SDP)** is a problem where we wish to minimize an objective function subject to constraints involving positive semidefinite matrices [12]. An SDP takes the form $\min \vec{c}^T \vec{x}$ s.t. $Q(x) = Q_0 + \sum_{i=1}^m x_i Q_i \succeq 0$, where $Q_0, \dots, Q_m \in \mathbb{R}^{n \times n}$ are PSD matrices.

Any PSD matrix $Q \in \mathbb{C}^{n \times n}$ can be rewritten in the form $Q = B^* B$ for some matrix B , where $*$ denotes the conjugate transpose. Therefore, we have for any vector $\vec{v} \in \mathbb{C}^{n \times n}$, we have $\vec{v}^* Q \vec{v} = \vec{v}^* B^* B \vec{v} = (B \vec{v})^* B \vec{v}$, which becomes an SOS when expanded. If $\vec{v}$ is a vector of monomials (single-term polynomials with coefficient 1), we can write the problem of finding the SOS decomposition of a polynomial $p + \lambda$ as the following SDP:

$$\min \lambda \text{ s.t. } p + \lambda = \vec{v}^* Q \vec{v}, Q \succeq 0, \lambda \geq 0$$

In order for equality to hold, $\vec{v}$ must contain the appropriate monomials to construct the sum of squares; if not, the problem will be infeasible and must be attempted again with different or possibly additional entries in $\vec{v}$, possibly of higher degrees.

Any SOS polynomial has a corresponding representation using PSD matrices. Before proving this result, we introduce some notation. Let $\mathbf{x} = (x_1, \dots, x_n)$ be a vector of monomials and $\gamma = (\gamma_1, \dots, \gamma_n)$ be a vector of natural numbers. Then, $\mathbf{x}^\gamma = x_1^{\gamma_1} \dots x_n^{\gamma_n}$, and $|\gamma|$ is the sum of the entries of γ . Let $\mathbb{N}_d^n$ indicate the set of n -dimensional vectors of natural numbers where the sum of the entries do not exceed d .

Theorem 1.1. [7] *Let $p \in \mathbb{R}[\mathbf{x}]$, $p = \sum_{\alpha \in \mathbb{N}_{2d}^n} p_\alpha \mathbf{x}^\alpha$ be a polynomial of degree $\leq 2d$. p is an SOS iff the following system in the matrix variable $X = (X_{\alpha, \beta})_{\alpha, \beta \in \mathbb{N}_d^n}$ is feasible:*

$$\begin{cases} X \succeq 0 \\ \sum_{\beta, \gamma \in \mathbb{N}_d^n | \beta + \gamma = \alpha} X_{\beta, \gamma} = p_\alpha \quad (|\alpha| \leq 2d) \end{cases} \quad (1)$$

Proof. Let $\mathbf{z}_d := (\mathbf{x}^\alpha \mid |\alpha| \leq d)$ denote the vector containing all monomials of degree at most d . For polynomials $u_j \in \mathbb{R}[\mathbf{x}]_d$, we have $u_j = (\vec{u}_j)^T \mathbf{z}_d$, where $\vec{u}_j$ is a vector containing the coefficients of the terms of u_j . Then, if p is an SOS, $p = \sum_j u_j^2 = \sum_j (\vec{u}_j^T \mathbf{z}_d)^2 = \sum_j \mathbf{z}_d^T \vec{u}_j \vec{u}_j^T \mathbf{z}_d =$

$$\mathbf{z}_d^T \left(\sum_j \vec{u}_j \vec{u}_j^T \right) \mathbf{z}_d = \mathbf{z}_d^T X \mathbf{z}_d, \text{ where } X \text{ is a PSD matrix. Solving for the coefficients yields (1). } \quad \square$$

There exist solvers that take SDPs as inputs and output the optimal objective value and variables; for example, in Python solvers include MOSEK, SCS, CVXOPT, and ECOS. By rewriting the SOS decomposition problem as an SDP, we can give the problem to such a solver and easily get our answer. Hence, our solver will be a program in Python that takes a polynomial as input from the user and uses SDPs, CVXPY, and convex solvers (specifically, SCS), to find the SOS decomposition of the polynomial.

2 Creating the solver

Our end goal is to create a non-commutative, constrained polynomial SOS solver. Rather than starting to develop this tool from scratch, we instead began by developing a very simple solver, starting with a commutative, unconstrained polynomial SOS solver, and slowly editing the solver until we get our final product. SymPy and CVXPY are required to run the solver; both can be installed by running “pip install sympy cvxpy” in the command line.

2.1 Commutative Unconstrained Solver

Our commutative unconstrained solver takes as input a list of variables, a polynomial p , and a vector of monomials $\vec{v}$, and outputs the minimum $\lambda \geq 0$ such that $p + \lambda$ is an SOS, the matrix Q that results in the minimum λ , and the SOS decomposition of $p + \lambda$. The solver can be run from the command line using the file `soassolvernoconstraints.py`.

Example 2.1. *Examples of the solver in use as seen from the command line:*

```
Enter the variables (comma-separated): x,y
Enter the monomials in terms of x,y (comma-separated with natural number exponents):
x,y,1
Enter a polynomial in terms of x,y: x^2-2xy+y^2-4
Q matrix: [[ 1. -1.  0.]
[-1.  1.  0.]
[ 0.  0.  0.]]
Lambda: 4.0
SOS decomposition: (-1.0*x + 1.0*y)**2.0

Enter the variables (comma-separated): x,y
Enter the monomials in terms of x,y (comma-separated with natural number exponents):
x,y,1
Enter a polynomial in terms of x, y: x^2-y^2
Problem is infeasible. Exiting.
```

The solver gets the variables and $\vec{v}$ using the function `get_monomial_vector` from the file `monomial_input.py`, and gets p using the function `get_polynomial_from_input` from the file `poly_input.py`. The solver works by computing the left- and right-hand sides of the equation $p + \lambda = \vec{v}^T Q \vec{v}$, and using the function `get_coeffs` from the file `get_coeffs.py` to equate the coefficients of each term to get a system of equations involving the entries of Q . This system of equations is passed to the convex solver as a set of constraints, then the solver solves for λ and Q such that λ is minimized. To get the SOS decomposition of p , the function `matrix_sqrt` from the file `matrix_sqrt.py` uses eigenvalue decomposition to find the matrix B such that $B^T B = Q$, then the solver outputs $B \vec{v} \cdot B \vec{v}$. Finding B is necessary as simply outputting $\vec{v}^T Q \vec{v}$ will not result in the correct form for an SOS decomposition.

Below is the pseudocode for the solver:

```
v = get_vector_from_input()
lm = Var("lm")
p = get_poly_from_input()+lm
Q = Var("Q", PSD = True)
vQv = v.T * Q * v
constraints = extract_coefficients(p,vQv)
Q, lm = solve(min lm subject to constraints)
```

However, an issue arises when extracting the coefficients of $p + \lambda$ and $\vec{v}^T Q \vec{v}$: as we wish for the solver to solve for λ and Q , we must create λ and Q as CVXPY variables in Python. Since the values of $\vec{v}$ and p are known, we create them as SymPy variables in Python. However, trying to multiply or compare coefficients of CVXPY and SymPy variables raises an error. We solve this problem by creating copies of λ and Q in SymPy, then using these SymPy copies to obtain our system of equations.

When we attempt to pass this system of equations to CVXPY, we run into another error: our system of equations involve the entries of our SymPy copy of Q , but CVXPY requires any constraints to be in terms of CVXPY variables. Therefore, we must parse our system of equations using the function `sp_to_cp_constraint` from the file `sp_to_cp_constraint.py` to replace all instances of SymPy variables with their corresponding CVXPY variables.

Most convex solvers rarely output exact integers, instead outputting values very close to integers but with lots of decimal places. To clean up the solver output, the function `clean_value` from the file `clean_value.py` rounds values that are sufficiently close to an integer given a tolerance (0.0001 by default), and rounds all other values to 2 decimal places.

2.2 Commutative Constrained Solver

Suppose we now wish to determine the minimum of p where we wish to restrict the feasible region only to inputs that satisfy $g \geq 0$; for example, consider $p = x^3$ such that $x \geq 0$ and $10 - x \geq 0$. The following result lets us find a certificate of non-negativity for a polynomial p in terms of these non-negativity constraints:

Theorem 2.1. Putinar's Positivstellensatz [10]: *Let $K = \{x \in \mathbb{R}^n : g_1(x) \geq 0, \dots, g_k(x) \geq 0\}$ be a compact semi-algebraic subset of $\mathbb{R}^n$ and let p be a positive polynomial on K . Then there exists sum-of-squares $s_0, \dots, s_k$ such that $p = s_0 + s_1g_1 + \dots + s_kg_k$*

Our solver solves for the sum-of-squares $s_0, \dots, s_1$ by creating a PSD matrix Q_i for each sum-of-squares s_i and solving the equation $p = \vec{v}^T Q_0 \vec{v} + \vec{v}^T Q_1 \vec{v} g_1 + \dots + \vec{v}^T Q_k \vec{v} g_k$. Note that from the theorem, the $\vec{v}$ associated with each Q_i does not necessarily need to be the same, but using the same $\vec{v}$ for each Q_i certainly makes things easier for the implementation and the user, and still yields the desired decomposition of p assuming $\vec{v}$ is sufficiently large. The solver can be run from the command line using the file `ssosolveryesconstraints.py`. A new function, `get_constraints` from the file `get_constraints.py`, is responsible for processing constraints from the command line.

Example 2.2. *Examples of the solver in use as seen from the command line:*

```
Enter the variables (comma-separated): x
Enter the monomials in terms of x (comma-separated with natural number exponents):
1,x
Enter a polynomial in terms of x: x^3
Enter ≥ 0 constraints g(x) (comma-separated) (press Enter to finish): x, 10-x
Lambda: 0
Q Matrices: [[0 0]
[0 0]]
[[1. 0.]
[0. 0.]]
[[0 0]
[0 0]]
SOS decomposition of p(x) + λ:
0*(10-x) + x**2.0*(x)
```

In the example above, we specifically added the constraint $10 - x \geq 0$ to demonstrate the requirement of a compact feasible region for Putinar’s Positivstellensatz. However, the solver may be able to find an SOS decomposition even if the feasible region is unbounded, although we do not guarantee this; for example, the above problem with only the constraint $x \geq 0$ still yields an SOS decomposition.

An issue arose when testing the solver on constraints with no feasible region, for example if given the constraints $x - 1 \geq 0$ and $-x - 1 \geq 0$, which has no solutions. Despite this, the solver would still output an SOS decomposition of p rather than returning infeasible. The reason was that Putinar’s Positivstellensatz assumes the constraints form a feasible region and that all constraint functions are nonnegative for all inputs. To fix this, the function `is_feasible` from the file `is_feasible.py` was added to verify the existence of a feasible region before deploying the solver. The file works by using SymPy’s boolean operator “And” to append all the inequality constraints

as a truth statement, then simplifies and check for a feasible region using `sp.reduce_inequalities` for univariate constraints, or SymPy’s “satisfiable” tool with the algorithm “dpll2” for multivariate constraints.

2.3 Commutative Complex Unconstrained Solver

Our previous two solvers only work for real polynomials, so if we wish to solve problems involving complex polynomials and matrices, we must adapt our solver as such. As Putinar’s Positivstellensatz only applies to real problems, we omit constraints for our complex solver.

For our complex polynomial we adapt our definition of non-negativity as such:

Definition 2.1. A complex polynomial $p \in \mathbb{R}[x_1, \dots, x_n]$ is non-negative if $\text{re}(p(x_1, \dots, x_n)) \geq 0$ and $\text{im}(p(x_1, \dots, x_n)) = 0 \forall x_1, \dots, x_n \in \mathbb{R}$.

Most common convex solvers only work with real problems, so we must rewrite our complex polynomial optimization problems as strictly real ones. If p is a complex polynomial, $\vec{v} \in \mathbb{R}^n$ is a vector of monomials, $Q \in \mathbb{C}^{n \times n}$ is a complex matrix, and $R, I \in \mathbb{R}^{n \times n}$ are matrices containing the real and imaginary parts of Q , respectively, then the complex SDP

$$\min \lambda \text{ s.t. } p + \lambda = \vec{v}^* Q \vec{v}, Q \succeq 0$$

is equivalent to the following real SDP [14]:

$$\min \lambda \text{ s.t. } \text{re}(p) + \lambda = \vec{v}^T R \vec{v}, \text{im}(p) = \vec{v}^T I \vec{v}, \begin{pmatrix} R & -I \\ I & R \end{pmatrix} \succeq 0$$

As the complex numbers are not an ordered field, in order to minimize λ we require λ to be real.

Example 2.3. Examples of the solver in use as seen from the command line:

```
Enter the variables (comma-separated): x
Enter the monomials in terms of x (comma-separated with natural number exponents):
1,x
Enter a polynomial in terms of x, I (use 'i' for  $\sqrt{-1}$ ): (x-i)^2+2xi+1
Lambda: 0.0
Q Matrices:
[[0.  0.  0.  0.]
 [0.  1.  0.  0.]
```



```
[0.  0.  0.  0.]
[0.  0.  0.  1.]]
SOS decomposition of p(x) + λ:  1.0*x**2.0
```

2.4 Non-Commutative Unconstrained Solver

Now we begin developing the solver that takes as input a non-commutative, matrix-valued polynomial. This polynomial takes matrices as inputs, and also outputs a matrix. Now our variables are matrices, and our monomial vector $\vec{v}$ is a vector whose entries are monomials in matrix variables. For matrix-valued polynomials, we must adapt our definition of a sum-of-squares and non-negative as follows:

Definition 2.2. A matrix-valued polynomial p is a **Hermitian sum-of-squares (SOS)** if there exist matrices $A_1, \dots, A_k$ such that $p = \sum_{i=1}^k A_i^* A_i$, where A_i^* indicates the conjugate transpose of A_i .

Definition 2.3. A matrix-valued polynomial is **non-negative** if, for any input matrices $X_1, \dots, X_i$, the matrix outputted by $p(X_1, \dots, X_i)$ is PSD.

Similarly, instances of $\vec{v}^T Q \vec{v}$ in the commutative case are replaced with $\vec{v}^* Q \vec{v}$, where $\vec{v}^*$ is equal to $\vec{v}$ but with entry replaced by its conjugate transpose. For example, if $\vec{v} = (AB \ CD)$, then $\vec{v}^* = \begin{pmatrix} (AB)^* \\ (CD)^* \end{pmatrix} = \begin{pmatrix} B^* A^* \\ D^* C^* \end{pmatrix}$.

Additionally, rather than solving for λ such that $p + \lambda$ is an SOS, we solve for λ such that $p + \lambda I$ is an SOS, where I is the identity matrix. In short, we solve for Q and λ in the equation $p + \lambda I = \vec{v}^* Q \vec{v}$.

Finally, in the function for taking user input for the monomial vector, `get_vars_vec` from the file `NCmonomial.input.py`, a feature was added where the user may input a natural number n , and a vector $\vec{v}$ will be generated whose entries are all possible monomials that can be made with the given variables up to degree n . For example, if the variables are A and B and the user inputs 2 when prompted for $\vec{v}$, $\vec{v}$ will become $[I \ A \ B \ A^2 \ AB \ BA \ B^2]$.

Note that the solver uses I to represent the identity matrix, as opposed to 1. The solver can be run from the command line using the file `NCsolvernoconstraints.py`

Example 2.4. Examples of the solver in use as seen from the command line:

```
Enter the matrix variable names (assumed to be Hermitian) (comma-separated):  A,B
Enter the monomial expressions (comma-separated; enter ^T for transpose, ^* for
conjugate transpose, AB for A*B): AB,I
Enter a polynomial in terms of B,I,A, Adjoint:  B^*A^*AB-4I
```

```

Lambda: 4.0
Q0 matrix = [[1.  0.]
[0.  0.]]
SOS decomposition: 1.0*Adjoint(B)*Adjoint(A)*A*B

```

Working with matrix variables caused some issues to arise when taking user input from the command line. For example, using variable names with multiple letters like A0 or B0 worked fine with the commutative solver, but when done in the non-commutative solver, an error would be raised or, depending on the context, the solver would sometimes misinterpret a variable like A0 as A^*0 . In the commutative solver, the SymPy parsing tool “implicit_multiplication_application” was used to parse entries of the monomial vector; for example, parsing A0B0 as $A0*B0$ in the commutative case. Strangely, the tool seems to malfunction for matrix variables, so a separate parsing function, `insert_multiplication` in the file `NCmonomial.input.py`, was created to add multiplication symbols inbetween adjacent variables.

Another problem occurred when trying to extract the coefficient from the left- and right-hand sides of the expression $p + \lambda I = \vec{v}^* Q \vec{v}$. In the commutative case, the left- and right-hand sides were converted to SymPy polynomials, and the system of equations of coefficients was derived using attributes and tools like `.monoms()`, `.coeff_monomial`, and `sp.solve`. However, these attributes and tools do not work for non-commutative variables like matrices.

To solve this problem, rather than equate coefficients between the left- and right-hand sides, we subtracted the right-hand side from the left-hand side, yielding $p + \lambda I - \vec{v}^* Q \vec{v} = 0$. We then wrote a function, `get_coeffs` in the file `NCget_coeffs.py`, that extracts the scalar and matrix part of each term, and sets all scalar parts to 0, hence giving us our system of equations.

2.5 Non-Commutative Constrained Solver

Finally, we have the non-commutative constrained solver that finds the SOS decomposition of a matrix-valued polynomial, subject to constraints. Recall Putinar’s Positivstellensatz for commutative variables, which stated non-negative polynomials could be written as a sum of sums-of-squares times the constraint functions under some compact condition. A similar result exists for non-commutative variables:

Theorem 2.2. Helton and McCullough’s Positivstellensatz for Non-Commutative Variables [4]: Suppose $\mathcal{P}$ is a subset of matrices with non-commutative polynomial entries and suppose that $\mathcal{D}_{\mathcal{P}}$, the set of inputs that make $\mathcal{P}$ PSD, is bounded. If q , a matrix with non-commutative polynomial entries, is strictly positive definite on $\mathcal{D}_{\mathcal{P}}$, then $q = \sum_1^N s_j^T p_j s_j + \sum_1^M r_k^T r_k$ for some

matrix polynomials s_j, p_j and r_k .

For our purposes, this means, if a matrix polynomial p is non-negative on a set of non-negativity constraints g_i , then it can be written in the form $p = \vec{v}^* Q_0 \vec{v} + \sum_i \vec{v}^* Q_i \vec{v} g_i \vec{v}^* Q_i \vec{v}$. Once again, our solver creates a PSD matrix Q_i for each sum-of-squares s_i . However, attempting to solve the problem using a similar equation to the commutative case ($p + \lambda I = \vec{v}^* Q_0 \vec{v} + \vec{v}^* Q_1 \vec{v} g_1 + \dots + \vec{v}^* Q_k \vec{v} g_k$) still works for certain problems (but not all) and is much less computationally expensive. Therefore, for the constrained case our solver first attempts to solve the aforementioned equation, and if that fails, only then does it resort to solving the more complex equation $p = \vec{v}^* Q_0 \vec{v} + \vec{v}^* Q_1 \vec{v} g_1 \vec{v}^* Q_1 \vec{v} + \dots + \vec{v}^* Q_k \vec{v} g_k \vec{v}^* Q_k \vec{v}$, using the function `vqvgvqv` in the file `vqvgvqv.py`.

In quantum information, non-commutative polynomial optimization is frequently used for quantum correlations defined by equality constraints, rather than inequality constraints [11]. However, as Helton and McCullough's Positivstellensatz only applies to non-negativity constraints, if we wish to enforce an equality-with-0 constraint, we must assert that the constraint and its negative are non-negative. For example, consider a polynomial that takes in a matrix A such that $A^2 = I$. The statement $A^2 = I$ is equivalent to $A^2 - I = 0$ which is equivalent to $A^2 - I \geq 0$ and $-A^2 + I = I - A^2 \geq 0$. Hence, our solver takes equality-with-0 constraints as input, creates a list of those constraints, and then appends the negative of each constraint to that list. Similarly, as all constraint functions g_k are equal to 0, our solver only prints $\vec{v}^* Q_0 \vec{v}$ in the SOS decomposition, as any terms $\vec{v}^* Q_k \vec{v} g_k$ or $\vec{v}^* Q_k \vec{v} g_k \vec{v}^* Q_k \vec{v}$ will simplify to 0.

One constraint that appears in these polynomial optimization problems when the output or the players are binary [1] is the constraint that the matrix variables are Hermitian, i.e. $A^* = A$. Hence, our solver assumes that all matrix variables are Hermitian.

Recall that our original intent behind building the solver was to find the maximum winning probability of a non-local game; however, the SOS decomposition of a polynomial p only tells us the minimum of p . Finding the maximum value of p is equivalent to finding the minimum value of $-p$. Therefore, we must instead find the SOS decomposition of $-p + \lambda I$ rather than $p + \lambda I$. Hence, our solver takes p as input, but outputs the SOS decomposition of $-p + \lambda I$ rather than the SOS decomposition of $p + \lambda I$.

Finally, in order to reduce the computational work of the solver, the solver simplifies the problem by using constraints involving the identity matrix. For example, consider a polynomial p takes in a matrix variable A with the constraint that $A^2 = I$. When expanding and simplifying the equation $p + \lambda I = \vec{v}^* Q_0 \vec{v} + \sum_{i=1}^k \vec{v}^* Q_i \vec{v} g_i$ (or $p + \lambda I = \vec{v}^* Q_0 \vec{v} + \sum_{i=1}^k \vec{v}^* Q_i \vec{v} g_i \vec{v}^* Q_i \vec{v}$), any instance of A^2 , or a power of A^2 like A^4 or A^6 , can be replaced with I , significantly simplifying the problem. Similarly, an instance of A^3 or A^5 could be replaced with A . This is done using the

functions `sub_identity_matrix` and `get_matrix_exps` from the file `sub_identity_matrix.py`.

The solver can be run from the command line using the file `NCsolverconstraints.py`. Below we demonstrate the use of the solver on the CHSH polynomial with constraints.

Example 2.5. *Examples of the solver in use as seen from the command line:*

```
Enter the matrix variable names (assumed to be Hermitian) (comma-separated): A0, A1,
B0, B1
Enter the monomial expressions (comma-separated; enter ^T for transpose, ^* for conjugate
transpose, AB for A*B): A0,A1,B0,B1
Enter a polynomial in terms of B0, A1, I, B1, A0, Adjoint: A0B0+A0B1+A1B0-A1B1
Enter = 0 constraints g(B0, A1, I, B1, A0)(comma-separated) (press Enter to finish):
I-A0^2,I-A1^2,I-B0^2,I-B1^2,A0B0-B0A0,A0B1-B1A0,A1B0-B0A1,A1B1-B1A1
Enter = I constraints g(B0, A1, I, B1, A0) (comma-separated) (press Enter to finish):
A0^2, A1^2, B0^2, B1^2
Lambda: 2.8284361370790756
Q0 matrix = [[ 0.70710904 0. -0.50000001 -0.50000001]
[ 0. 0.70710904 -0.50000001 0.50000001]
[-0.50000001 -0.50000001 0.70710904 0. ]
[-0.50000001 0.50000001 0. 0.70710904]]
SOS decomposition: (-0.819839892717626*Adj(B0) - 0.18700392378133*Adj(B1))*
(-0.819839892717626*B0 - 0.18700392378133*B1) + (-0.711946090242157*Adj(B0)
+ 0.447482605013613*Adj(B1))* (-0.711946090242157*B0 + 0.447482605013613*B1)
+ (0.18700392378133*Adj(B0) - 0.81983989271763*Adj(B1)) *(0.18700392378133*B0
- 0.81983989271763*B1) + (0.447482605013614*Adj(B0) + 0.711946090242158*Adj(B1))*
(0.447482605013614*B0 + 0.711946090242158*B1)
```

We can retrieve the exact value of λ , which is $2\sqrt{2}$. From this, we can find the optimal win probability of the CHSH game p , represented by $p = \frac{1}{2} + \frac{\lambda}{8}$ [2]. When we substitute $\lambda = 2\sqrt{2}$, we obtain $p \approx 85\%$.

Originally in the solver output, terms multiplied by zero would not be reduced automatically, so the output was cluttered with many terms like $0 \cdot A$ or $0 \cdot \text{Adjoint}(B)$. To remedy this, the function `remove_zero_terms` in the file `remove_zero_terms.py` was created, containing a function that takes the final SOS decomposition as input, and returns the same SOS decomposition but with all terms multiplied by 0 removed.

3 Areas of Improvement

There are a few improvements that could be made to further develop the solver. The first is would be to expand the constraint substitution feature of the solver. The solver is already equipped to substitute identity matrix constraints using the function `sub_identity_matrices` in the file `sub_identity_matrices.py`. However, the solver could be improved if we expand this feature by allowing any constraint to be substituted [5]. For example, if A and B are matrix variables

and we assert that $AB = BA$, then, when solving the equation $p + \lambda I = \vec{v}^* Q_0 \vec{v} + \sum_{i=1}^k \vec{v}^* Q_i \vec{v} g_i$

(or $p + \lambda I = \vec{v}^* Q_0 \vec{v} + \sum_{i=1}^k \vec{v}^* Q_i \vec{v} g_i \vec{v}^* Q_i \vec{v}$), any instance of AB would be replaced with BA ,

hopefully simplifying the equation and the coefficient extraction process. In general, any instance of the left-hand side of the constraint is replaced with the right-hand side of the constraint. The constraint substitution could perhaps be implemented as a loop, simplifying the equation until no further substitutions could be made. The user would need to ensure that none of the constraints passed to the solver are “circular”; for example, if the user asserts that $A = B$ and $B = A$, an infinite loop could arise.

We also wish to retrieve the exact values of λ and the entries of Q using symbolic computation. This can be done with an algorithm called the *LLL* algorithm [6], which replaces each numeric value with its most likely symbolic representation by solving for the roots of a given polynomial. Although this algorithm does not always work, it does work in some instances.

Additionally, there exists a tool called the Newton polytope that can find and eliminate some of the unnecessary monomial entries of $\vec{v}$, reducing the size of $\vec{v}$ while still maintaining the SOS decomposition of p [8].

Finally, another way we can optimize our solver is by simplifying the system of equations we pass to our convex solver. This system of equations involves the entries of the Q matrices, but using a tool called the Gröbner basis, we can rewrite this system of equations such that the solution set is the same, but the number of equations in the system of equations is less [13], hopefully reducing the computational work of the convex solver.

4 Acknowledgements

We acknowledge the support of the Natural Sciences and Engineering Research Council of Canada (NSERC)(ALLRP-578455-2022) as well as the support of the Air Force Office of Scientific Research under award number FA9550-20-1-0375. The supervision of Prof. Anne Broadbent and Dr. Denis Rochette, both of the University of Ottawa, was immensely helpful throughout the completion of

this project.

References

- [1] Sabine Burgdorf, Igor Klep, and Janez Povh. *Optimization of Polynomials in Non-Commuting Variables*. Springer International Publishing, 2016.
- [2] B. S. Cirel’son. Quantum generalizations of bell’s inequality. *Letters in Mathematical Physics*, 4(2):93–100, March 1980.
- [3] John F. Clauser, Michael A. Horne, Abner Shimony, and Richard A. Holt. Proposed experiment to test local hidden-variable theories. *Physical Review Letters*, 23(15):880–884, October 1969.
- [4] J. Helton and Scott McCullough. A positivstellensatz for non-commutative polynomials. *Transactions of the American Mathematical Society*, 356(9):3721–3737, March 2004.
- [5] Marie Ioannou and Denis Rosset. Noncommutative polynomial optimization under symmetry, 2021.
- [6] Alex Kalbach and Ted Chinburg. Lll algorithm for lattice basis reduction, 2024.
- [7] Monique Laurent. *Sums of Squares, Moment Matrices and Optimization Over Polynomials*, page 157–270. Springer New York, September 2008.
- [8] Johan Lofberg. Pre- and post-processing sum-of-squares programs in practice. *IEEE Transactions on Automatic Control*, 54(5):1007–1011, 2009.
- [9] Hamoon Mousavi, Seyed Sajjad Nezhadi, and Henry Yuen. Nonlocal games, compression theorems, and the arithmetical hierarchy. 2021.
- [10] Mihai Putinar. Positive polynomials on compact semi-algebraic sets. *Indiana University Mathematics Journal*, 42(3):969, 1993.
- [11] Armin Tavakoli, Alejandro Pozas-Kerstjens, Peter Brown, and Mateus Araújo. Semidefinite programming relaxations for quantum correlations. *Reviews of Modern Physics*, 96(4), December 2024.
- [12] Lieven Vandenbergh and Stephen Boyd. Semidefinite programming. *SIAM Review*, 38(1):49–95, March 1996.
- [13] Joris van der Hoeven and Robin Larrieu. Fast gröbner basis computation and polynomial reduction for generic bivariate ideals. *Applicable Algebra in Engineering, Communication and Computing*, 30(6):509–539, June 2019.
- [14] Jie Wang. A more efficient reformulation of complex sdp as real sdp, 2023.